

■ FinanceAI

AI-Powered Accounting SaaS

Complete MVP Blueprint & Technical Architecture

30-Day Execution Plan · Supabase + Node.js + Next.js · AI by Gemma

Version 2.0 · June 2026 · Confidential

\$47B+	150M+	30 Days	Gemma
Market Size	SMBs Worldwide	MVP Deadline	AI Engine

Table of Contents

Section 1	Product Vision & Market Opportunity	3
Section 2	SaaS Business Model & Pricing Tiers	4
Section 3	MVP Feature Prioritization (Gemma-Mapped)	6
Section 4	Gemma AI Engine — Architecture & Feature Design	8
Section 5	Innovative AI Features (Gemma-Powered)	10
Section 6	Database Schema Design	11
Section 7	Multi-Tenant SaaS Architecture	13
Section 8	Supabase Architecture & RLS Policies	14
Section 9	Backend Architecture (Node.js)	15
Section 10	REST API Design	16
Section 11	Frontend Architecture (Next.js)	17
Section 12	Security & Compliance	18
Section 13	4-Week Development Roadmap	19
Section 14	DevOps & Deployment	21
Section 15	Scaling Strategy	22
Section 16	Launch Strategy	23

1. Product Vision & Market Opportunity

Why FinanceAI will disrupt the \$47B accounting software market

The Problem

Small businesses and freelancers waste an average of **10+ hours per week** on manual bookkeeping, invoice management, and financial reconciliation. Existing tools like QuickBooks and Xero are powerful but complex, expensive, and built for accountants — not for the business owner who just wants to know if their company is healthy. The gap: **no AI-first, simple, affordable accounting platform exists for the 150M+ SMBs globally.**

Market Opportunity

Segment	Size	CAGR	Addressable
Global Accounting Software	\$47.6B (2025)	8.5%	High
SMB FinTech SaaS	\$22.1B (2025)	12.3%	Very High
AI in Finance	\$15.8B (2025)	23.4%	Emerging
Freelancer Tools Market	\$6.2B (2025)	18.1%	High

Target Audience

Freelancers & Solopreneurs	Need simple invoicing, expense tracking, and tax prep — not enterprise complexity.
Small Businesses (1-50 staff)	Need multi-user access, payroll basics, and real-time cash flow visibility.
Startups	Need investor-ready reports, burn rate tracking, and runway forecasting.
SMEs (51-500 staff)	Need team collaboration, approval workflows, and advanced reporting.
Accountants & Bookkeepers	Need white-label tools to serve multiple clients from one dashboard.

Competitive Positioning

Feature	FinanceAI	QuickBooks	Xero	FreshBooks
AI CFO Assistant	■ Native	■ None	■ None	■ None
Natural Language Queries	■ GPT-4	■	■	■
Receipt OCR	■ Built-in	■ Add-on	■ Add-on	■ Add-on
Fraud Detection	■ ML-based	■	■	■
Cash Flow Forecast	■ AI	■ ■ Basic	■ ■ Basic	■
Starting Price/mo	\$0 (Free tier)	\$30	\$13	\$17

Feature	FinanceAI	QuickBooks	Xero	FreshBooks
Setup Complexity	5 min	Hours	Hours	30 min
Multi-tenant (Accountants)	■ Native	■■■ ProAdvisor	■■■ Partner	■
API Access	■ All plans	■ Paid only	■ Paid only	■

Unique Value Proposition: "FinanceAI is the only accounting platform where your AI CFO works 24/7 — categorizing expenses, forecasting cash flow, detecting anomalies, and answering financial questions in plain English. Built for founders who want insights, not spreadsheets."

2. SaaS Business Model & Pricing Tiers

5-tier subscription model with premium add-ons and upsell triggers

	FREE TRIAL	STARTER	GROWTH	PROFESSIONAL	ENTERPRISE
Monthly Price	\$0	\$19	\$49	\$99	Custom
Annual Price	—	\$15/mo	\$39/mo	\$79/mo	Custom
Trial Duration	14 days	Ongoing	Ongoing	Ongoing	Ongoing
Users / Seats	1	1	3	10	Unlimited
Invoices/mo	5	50	Unlimited	Unlimited	Unlimited
Expenses/mo	20	200	Unlimited	Unlimited	Unlimited
Storage	100 MB	1 GB	10 GB	50 GB	1 TB+
AI Credits/mo	50	500	2,000	10,000	Unlimited
Receipt OCR	5/mo	50/mo	500/mo	Unlimited	Unlimited
Reports	Basic	Standard	Advanced	Full Suite	Custom
API Access	■	Read-only	Full	Full + Webhooks	Enterprise API
Team Collab	■	■	■	■ + Approvals	■ + SSO
AI CFO Chat	■	Basic	Advanced	Full Memory	Dedicated
Cash Forecast	■	■	3-month	12-month + AI	Custom
Fraud Detection	■	■	■	■ Real-time	■ Custom ML
Priority Support	■	Email	Email+Chat	Phone+Chat	Dedicated CSM
White Label	■	■	■	■	■

Premium Add-Ons

Add-On	Price	Description	Upsell Target
Extra AI Credits (5,000)	\$19/mo	For heavy AI users hitting monthly quota	Starter, Growth
OCR Pack (1,000 receipts)	\$29/mo	Bulk receipt scanning for expense-heavy businesses	All tiers
Payroll Module	\$39/mo + \$4/employee	Basic payroll runs, tax withholding, pay slips	Growth+
Tax Filing Automation	\$49/mo or \$149/yr	Auto-generate tax returns, connect to tax APIs	Professional+
White-Label Mode	\$199/mo	Custom branding for accountant firms serving clients	Enterprise
Dedicated AI Training	\$499 one-time	Train AI on your chart of accounts & business rules	Enterprise
Priority Support SLA	\$99/mo	2-hour response, dedicated Slack channel	Professional+

Add-On	Price	Description	Upsell Target
Advanced Integrations	\$29/mo	Stripe, Shopify, PayPal, Plaid bank sync	Growth+

Upsell Strategy

- Usage-based upgrade prompts: Show a banner when user reaches 80% of quota
- Feature preview: Let free users preview Pro features, then gate behind upgrade
- Team invite as upsell: 'Invite a colleague' requires Growth tier or above
- AI credit exhaustion: When AI credits run out, offer instant top-up or upgrade
- Annual discount: Offer 20% discount prominently — increases LTV and reduces churn
- Accountant partner program: Accountants get free Professional access if they bring 5+ clients
- White-label as anchor: Charge \$199/mo but worth \$2000+/mo in client billing for agencies

3. MVP Feature Prioritization — Gemma AI Mapped

Three-tier roadmap with explicit Gemma AI role per feature

Gemma AI Philosophy: For this platform, Gemma acts as an intelligent assistant layer — not a replacement for solid database-driven accounting. The database does the heavy lifting for ledgers, invoices, and reports. Gemma enhances the UX by interpreting messy data, answering questions in plain English, and generating narrative insights. This separation keeps the MVP stable and auditable.

MUST-HAVE FEATURES — Month 1 (Core MVP)

These are non-negotiable. The database and standard backend do the heavy lifting for financial data integrity. Gemma is selectively integrated only where it adds **immediate, day-one value** without risk to data accuracy.

Feature	Who Does the Work	Gemma's Role	Value	Decision
Supabase Auth (OAuth + Email)	Supabase Auth	None — pure infra	Critical	■ SHIP
Organization + Multi-tenant	PostgreSQL + RLS	None — pure DB	Critical	■ SHIP
Core Ledger (Chart of Accounts)	PostgreSQL CRUD	None — data integrity first	Critical	■ SHIP
Invoice Generation + PDF	Node.js + DB	None — deterministic output	High	■ SHIP
Revenue & Expense Tracking	PostgreSQL CRUD	None — raw data capture	High	■ SHIP
Automated Expense Categorization	DB + Gemma	Gemma cleans bank descriptions (e.g. 'ADOBE *SYSTEMS INC' → 'Software Subscriptions'). Immediate, massive UX win.	High	■ SHIP w/ Gemma
Dashboard (KPI overview)	DB aggregation queries	None — pure SQL aggregates	High	■ SHIP
Basic Reports (P&L, Cash Flow)	PostgreSQL views	None — numbers must be exact	High	■ SHIP
Subscription Billing (Stripe)	Stripe SDK	None — payment infrastructure	Critical	■ SHIP
Receipt Upload + Storage	Supabase Storage	None at MVP — OCR is Month 2	High	■ SHIP
Role-Based Access (RBAC)	PostgreSQL + Middleware	None — security is non-negotiable	High	■ SHIP
Notification System (in-app)	PostgreSQL + Realtime	None — event-driven triggers	Med	■ SHIP

SHOULD-HAVE FEATURES — Month 2-3 (Competitive Edge)

Once the core web app handles transactions smoothly, these Gemma-powered features make FinanceAI stand apart from legacy accounting tools. They bridge complex financial data and non-accountant users.

Feature	Gemma's Specific Role	Why Now (Not MVP)	Timeline
Natural Language Data Retrieval	User types 'Show my net profit from last month' — Gemma parses intent, generates query parameters, DB executes. No menus needed.	Requires stable data schema first. Can't query what isn't clean.	■ Month 2
Invoice Data Extraction (OCR Post-Processing)	Standard OCR (Tesseract/Google Vision) extracts raw text from receipt PDF. Gemma then structures it into Vendor, Subtotal, Tax, Total fields.	OCR library ships first; Gemma layer added once upload flow is stable.	■ Month 2
Bank Account Sync (Plaid)	Plaid delivers raw transactions. Gemma auto-categorizes merchant names using your org's existing category patterns.	Plaid integration is high complexity — needs stable DB first.	■ Month 2
Email Invoice Reminders (Smart)	Gemma drafts tone-appropriate reminder copy based on how many days overdue and client payment history.	Requires invoice aging data — only meaningful after Month 1 usage.	■ Month 2
Expense Approval Workflows	No Gemma role — pure workflow logic with DB state machine.	UX complexity — ship after core expense flow is proven.	■ Month 2
Multi-currency Support	No Gemma role — FX API + DB conversion logic.	Scope risk for MVP — add once base currency flow is solid.	■ Month 2
Client Portal (invoice view)	No Gemma role — read-only authenticated view.	Separate auth context — ship after internal flows are stable.	■ Month 2
Audit Trail / Logs UI	No Gemma role — DB audit_logs table already populated.	Low priority for users; high value for compliance buyers later.	■ Month 2

FUTURE FEATURES — Month 4+ (Scale & Expansion)

These require higher Gemma context window usage (increasing per-call costs), mature historical data, and stricter regulatory guardrails. Save them for after you have active paying users and stable infrastructure.

Feature	Gemma's Role	Why Deferred	Risk
Financial Summary & Insights (Virtual CFO)	Pull 3-12 months of transactions, generate plain-English monthly summaries, flag anomalies. Gemma acts as narrative engine.	Requires large context windows + historical data volume. Infrastructure must be mature.	Cost, Context Length
Intelligent AR Automation	Gemma drafts escalating email sequences for overdue invoices — adjusts tone based on days overdue and client relationship history.	Needs invoice aging patterns. Tone calibration requires real user feedback loop.	Tone Risk, Compliance
Context-Aware Tax & Compliance Assistant	Gemma answers local tax deduction questions and flags compliance gaps based on jurisdiction.	Tax laws change constantly. High legal liability. Requires fine-tuning, disclaimers, and legal review.	Legal Liability, Accuracy
Cash Flow Forecasting (AI Narrative)	Gemma interprets 90-day statistical forecast and narrates it: 'Based on your patterns, expect a cash dip in Week 3 of August.'	Requires sufficient transaction history (3+ months) for meaningful patterns.	Data Dependency
White-label mode for accountant firms	None — branding config	Enterprise sales cycle needed	Low
Mobile app with voice input	Whisper API for voice-to-expense logging	Platform complexity post-web	Med
Tax filing automation	None — third-party API (TaxJar/Avalara)	Regulatory partnerships required	High

4. Gemma AI Engine — Architecture & Feature Design

How Gemma integrates into the FinanceAI stack as a local or API-hosted model

Why Gemma?

Gemma (Google DeepMind) is a family of lightweight, open-weight language models optimized for on-device and server-side inference. For a financial SaaS MVP, Gemma offers three critical advantages over closed models:

Cost control: Run Gemma locally or self-host on Railway/GCP — no per-token API cost surprises as usage scales. Critical when AI credits are a metered billing feature.

Data privacy: Financial data never leaves your infrastructure if Gemma runs on-premise. Crucial for enterprise and regulated customers.

Fine-tunability: Gemma can be fine-tuned on your domain-specific accounting data (transaction descriptions, chart of accounts) for higher accuracy than a generic model.

Speed: Gemma 2B/7B runs fast enough for real-time categorization and NL search without noticeable latency in the UI.

Gemma Deployment Options

Option	Method	Cost Model	Recommended For
Ollama (Local Dev)	Run gemma:7b via Ollama on dev machine	Free	Dev / Testing only
Google AI Studio API	google-generativeai SDK, hosted by Google	Pay-per-token	MVP launch — fastest to ship
Vertex AI (GCP)	Managed Gemma endpoint on GCP	Per-request + infra	Growth phase — SLA guaranteed
Self-hosted on Railway	Ollama container on Railway private network	Fixed infra cost	Scale phase — cost predictable
Fine-tuned on Vertex AI	Custom Gemma fine-tuned on accounting corpus	Training + inference	Enterprise — highest accuracy

Gemma Feature Implementations

Feature A: Automated Expense & Transaction Categorization (Must-Have)

This is the highest-ROI Gemma integration for Month 1. Bank transaction descriptions are notoriously messy ('SQ *COFFEEBEAN 04-291', 'AMZN MKTP US*ZA39X'). Gemma normalizes them into clean categories using few-shot prompting with the org's own chart of accounts as context.

```
// Node.js – Expense Categorization Service
```

```
// src/ai/categorizer.ts

import { GoogleGenerativeAI } from "@google/generative-ai";

const genAI = new GoogleGenerativeAI(process.env.GEMMA_API_KEY!);
const model = genAI.getGenerativeModel({ model: "gemma-3-27b-it" });

interface CategoryResult {
  category: string;
  confidence: "high" | "medium" | "low";
  cleanName: string;
}

export async function categorizeTransaction(
  rawDescription: string,
  orgCategories: string[], // from org's chart of accounts
  recentExamples: Array<{ description: string; category: string }>
): Promise {
  const examples = recentExamples
    .slice(0, 5)
    .map(e => `${e.description}" → ${e.category}`)
    .join("\n");

  const prompt = `You are an accounting assistant. Categorize this bank transaction.

  ORGANIZATION CATEGORIES: ${orgCategories.join(", ")}

  RECENT EXAMPLES FROM THIS ORG:

  ${examples}

  TRANSACTION TO CATEGORIZE: "${rawDescription}"

  Respond in JSON only:

  {
    "category": "",
    "cleanName": "",
    "confidence": "high|medium|low"
  }`;

  const result = await model.generateContent(prompt);
  const text = result.response.text().trim();
  return JSON.parse(text.replace(/\\`\\`json|\\`\\`/g, ""));
}
```

Feature B: Natural Language Data Retrieval (Should-Have)

Users type a plain-English query. Gemma translates it into structured filter parameters. The backend executes a parameterized SQL query. No SQL injection risk — Gemma only outputs a JSON filter object, never raw SQL.

```
// Natural Language → Structured Filters
// src/ai/nlSearch.ts
```

```

const NL_SEARCH_PROMPT = `You convert accounting queries into JSON filter objects.

SCHEMA: expenses(amount, category, vendor, expense_date, status)
invoices(total, client_name, status, due_date, paid_at)
revenues(amount, source, revenue_date)

USER QUERY: "{query}"

Return ONLY valid JSON (no prose):
{
  "table": "expenses|invoices|revenues",
  "filters": {
    "dateFrom": "YYYY-MM-DD or null",
    "dateTo": "YYYY-MM-DD or null",
    "amountMin": number or null,
    "amountMax": number or null,
    "category": "string or null",
    "status": "string or null"
  },
  "orderBy": "field ASC|DESC",
  "limit": number
}`;

export async function parseNLQuery(userQuery: string) {
  const prompt = NL_SEARCH_PROMPT.replace("{query}", userQuery);
  const result = await model.generateContent(prompt);
  const json = result.response.text().replace(/\\\`\\\`json\\\`\\\`/g, "").trim();
  return JSON.parse(json); // backend then builds parameterized query from this
}

```

Feature C: Invoice/Receipt OCR Post-Processing (Should-Have)

Standard OCR (Tesseract, Google Cloud Vision) extracts raw text from uploaded receipt images and PDFs. The raw text is unstructured. Gemma's role is purely **post-processing** — parsing the raw OCR output into clean, structured expense fields.

```

// OCR Post-Processing Pipeline
// src/ai/ocrParser.ts

// Step 1: Extract raw text via Tesseract (or Google Vision)
// const rawText = await runOCR(imageBuffer); // "TOTAL $47.23\nTAX $3.82\nSUBTOT..."

// Step 2: Gemma structures the raw text
export async function parseReceiptText(rawOCRText: string): Promise {
  const prompt = `Extract structured data from this receipt OCR text.

RAW TEXT:
${rawOCRText}

Return ONLY valid JSON:

```

```
{
  "vendor": "string",
  "date": "YYYY-MM-DD or null",
  "subtotal": number or null,
  "taxAmount": number or null,
  "total": number,
  "currency": "USD",
  "lineItems": [{ "description": "string", "amount": number }],
  "receiptType": "restaurant|travel|software|office|other"
}`;

const result = await model.generateContent(prompt);
const text = result.response.text().replace(/\\`\\`\\`json\\`\\`\\`/g, "").trim();
return JSON.parse(text);
}

// Step 3: Auto-fill expense form with parsed data
// The frontend receives ReceiptData and pre-populates all fields
// User reviews and confirms – Gemma never auto-submits without human approval
```

Feature D: Financial Summary & CFO Insights (Future — Month 4+)

Gemma generates monthly plain-English financial summaries by ingesting aggregated transaction data. Context window usage is high (large financial history), so this is deferred until infrastructure is stable and cost-per-call is understood.

```
// Virtual CFO Monthly Summary
// Runs as a monthly cron job via BullMQ

export async function generateMonthlySummary(orgId: string, month: string) {
  // Pull aggregated data from DB (NOT raw transactions – too large)
  const data = await getMonthlyAggregates(orgId, month);
  // data = { totalRevenue, totalExpenses, netProfit, topExpenseCategories,
  // overdueInvoices, cashBalance, vsLastMonth }

  const prompt = `You are the CFO of ${data.orgName}. Write a concise monthly summary.

  FINANCIAL DATA FOR ${month}:
  - Revenue: ${data.totalRevenue}
  - Expenses: ${data.totalExpenses}
  - Net Profit: ${data.netProfit} (${data.profitMargin}% margin)
  - vs Last Month: ${data.revenueChange}% revenue change
  - Top Expense: ${data.topCategory} (${data.topCategoryAmount})
  - Overdue Invoices: ${data.overdueCount} totaling ${data.overdueAmount}
  - Cash Balance: ${data.cashBalance}

  Write a 3-paragraph summary: (1) overall health, (2) key concerns, (3) one recommendation.
  Be direct. Use plain English. No jargon.`;

  return await model.generateContent(prompt);
```

```
}
```

Feature E: Intelligent AR Automation (Future — Month 4+)

Gemma drafts tailored invoice reminder emails based on invoice age, amount, and client payment history. Each email is reviewed by the user before sending — Gemma never sends autonomously. Deferred because tone calibration requires real user feedback.

```
// AR Email Draft Generator
export async function draftInvoiceReminder(invoice: Invoice, daysOverdue: number) {
  const tone = daysOverdue < 7 ? "friendly" :
  daysOverdue < 21 ? "firm but professional" : "urgent";

  const prompt = `Draft an invoice payment reminder email.

  CONTEXT:
  - Client: ${invoice.clientName}
  - Invoice #: ${invoice.invoiceNumber}
  - Amount Due: $$${invoice.total} ${invoice.currency}
  - Days Overdue: ${daysOverdue}
  - Tone: ${tone}

  Write a concise email (subject + body). Do not threaten legal action.
  Do not invent payment details. Sign as "The FinanceAI Team".`;

  // Returns draft for USER REVIEW — never auto-sends
  return await model.generateContent(prompt);
}
```

Feature F: Tax & Compliance Assistant (Future — Month 4+)

Deferred — High Legal Risk. Tax law Q&A; requires jurisdiction-specific accuracy, changes frequently, and carries potential liability if wrong. Ship this only after: (1) consulting a tax attorney on disclaimer language, (2) restricting to general guidance (not specific advice), (3) adding mandatory 'consult a CPA' footer to every response, and (4) fine-tuning Gemma on verified tax content for your target jurisdictions.

5. Innovative AI Features — Gemma-Powered

Platform differentiators mapped to Gemma's actual capabilities

#	Feature	Gemma's Role & How	User Value	Difficulty	Revenue	Tier
1	Auto Expense Categorization	Gemma cleans raw bank descriptions into chart-of-accounts categories. Learns org-specific patterns via few-shot examples.	Extreme	Low	Extreme	Must-Have
2	Natural Language Search	Type 'Show marketing spend Q1' — Gemma extracts intent into JSON filters, DB executes. Zero menu navigation.	Very High	Low-Med	Very High	Should-Have
3	Receipt OCR Post-Processing	Tesseract extracts text; Gemma structures vendor/amount/tax/total into clean expense fields from the raw OCR output.	Very High	Med	High	Should-Have
4	Smart Invoice Reminder Drafts	Gemma drafts tone-calibrated reminder emails (friendly → firm → urgent) based on invoice age. User approves before send.	High	Low	High	Should-Have
5	Monthly CFO Summary	Gemma generates plain-English monthly P&L; narrative with anomaly flags and one actionable recommendation.	Very High	Med	Very High	Future
6	Business Health Score Narrative	DB computes 0-100 score from 8 metrics; Gemma writes the 2-sentence explanation in plain English.	High	Low	High	Should-Have
7	Smart Onboarding Wizard	Gemma asks 5 questions and recommends chart of accounts configuration, expense categories, and tax settings.	High (activation)	Low	High	Must-Have
8	Burn Rate Plain-English Alert	DB calculates runway months; Gemma narrates 'At this burn, you have 6.2 months of runway — consider cutting [top expense].'	Very High (startups)	Low	High	Should-Have
9	Vendor Name Normalization	Gemma standardizes inconsistent vendor names across imported transactions for clean reporting.	High	Low	Med	Must-Have
10	Context-Aware Tax Assistant	Gemma answers general tax deduction questions with mandatory CPA disclaimer. Requires fine-tuning and legal review.	Very High	High	Very High	Future
11	Anomaly Detection Narrative	DB detects statistical outliers (z-score); Gemma explains 'Your software expenses jumped 340% this month vs. average.'	High	Med	High	Future
12	Cash Flow Forecast Narrative	Stats model predicts 90-day cash; Gemma narrates the forecast with risk flags in plain English.	Very High	Med-High	High	Future
13	Scenario Planning Q&A;	User asks 'What if I hire 2 staff at \$5k/month?' — Gemma recalculates runway and narrates impact.	High	High	Very High	Future

#	Feature	Gemma's Role & How	User Value	Difficulty	Revenue	Tier
1 4	Duplicate Payment Detection	DB flags duplicate amounts/vendors within 30 days; Gemma generates a clear alert message for the user.	High	Low	High	Should-Have
1 5	WhatsApp Invoice Delivery	Invoice PDF sent via WhatsApp Business API. Gemma drafts the accompanying message based on client and invoice context.	High	Med	Med	Should-Have

6. Database Schema Design

Complete PostgreSQL multi-tenant schema with RLS, indexes, and constraints

Core Tables Overview

Table	Purpose
users	Auth identities, profile, preferences, tier
organizations	Tenant root — each company/business is one org
memberships	Many-to-many: user ↔ org with role assignment
subscriptions	Stripe subscription state per organization
plans	Product plans and feature quotas
invoices	Generated invoices with line items (JSON)
expenses	Expense records with receipt attachment
revenues	Income / revenue transactions
reports	Cached/generated financial report snapshots
notifications	In-app notification queue per user
usage_metrics	API calls, AI credits, storage per org per month
audit_logs	Immutable event log for compliance
ai_conversations	Chat history for AI CFO per organization
bank_accounts	Connected accounts via Plaid

Key SQL: Core Tables

```
-- USERS (managed by Supabase Auth, extended here)
CREATE TABLE public.users (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT UNIQUE NOT NULL,
  full_name TEXT,
  avatar_url TEXT,
  timezone TEXT DEFAULT 'UTC',
  language TEXT DEFAULT 'en',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ORGANIZATIONS (tenant root)
CREATE TABLE public.organizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

name TEXT NOT NULL,
slug TEXT UNIQUE NOT NULL,
owner_id UUID NOT NULL REFERENCES public.users(id),
plan_id UUID REFERENCES public.plans(id),
logo_url TEXT,
currency TEXT DEFAULT 'USD',
fiscal_year_end INTEGER DEFAULT 12, -- month number
tax_id TEXT,
country TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_orgs_owner ON public.organizations(owner_id);
CREATE INDEX idx_orgs_slug ON public.organizations(slug);

```

```

-- MEMBERSHIPS (RBAC)
CREATE TABLE public.memberships (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
organization_id UUID NOT NULL REFERENCES public.organizations(id) ON DELETE CASCADE,
user_id UUID NOT NULL REFERENCES public.users(id) ON DELETE CASCADE,
role TEXT NOT NULL CHECK (role IN ('owner', 'admin', 'accountant', 'manager', 'employee')),
invited_by UUID REFERENCES public.users(id),
joined_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(organization_id, user_id)
);
CREATE INDEX idx_memberships_org ON public.memberships(organization_id);
CREATE INDEX idx_memberships_user ON public.memberships(user_id);

-- SUBSCRIPTIONS
CREATE TABLE public.subscriptions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
organization_id UUID UNIQUE NOT NULL REFERENCES public.organizations(id),
stripe_customer_id TEXT UNIQUE,
stripe_subscription_id TEXT UNIQUE,
plan_id UUID REFERENCES public.plans(id),
status TEXT DEFAULT 'trialing'
CHECK (status IN ('trialing', 'active', 'past_due', 'canceled', 'paused')),
current_period_start TIMESTAMPTZ,
current_period_end TIMESTAMPTZ,
cancel_at_period_end BOOLEAN DEFAULT FALSE,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- INVOICES

```

```

CREATE TABLE public.invoices (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES public.organizations(id) ON DELETE CASCADE,
  invoice_number TEXT NOT NULL,
  client_name TEXT NOT NULL,
  client_email TEXT,
  line_items JSONB NOT NULL DEFAULT '[]',
  subtotal DECIMAL(15,2) NOT NULL DEFAULT 0,
  tax_rate DECIMAL(5,2) DEFAULT 0,
  tax_amount DECIMAL(15,2) DEFAULT 0,
  total DECIMAL(15,2) NOT NULL DEFAULT 0,
  currency TEXT DEFAULT 'USD',
  status TEXT DEFAULT 'draft'
  CHECK (status IN ('draft', 'sent', 'viewed', 'paid', 'overdue', 'cancelled')),
  due_date DATE,
  paid_at TIMESTAMPTZ,
  notes TEXT,
  pdf_url TEXT,
  created_by UUID REFERENCES public.users(id),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(organization_id, invoice_number)
);

CREATE INDEX idx_invoices_org ON public.invoices(organization_id);
CREATE INDEX idx_invoices_status ON public.invoices(organization_id, status);
CREATE INDEX idx_invoices_due ON public.invoices(due_date) WHERE status != 'paid';

-- EXPENSES
CREATE TABLE public.expenses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES public.organizations(id) ON DELETE CASCADE,
  amount DECIMAL(15,2) NOT NULL,
  currency TEXT DEFAULT 'USD',
  category TEXT NOT NULL,
  vendor TEXT,
  description TEXT,
  expense_date DATE NOT NULL DEFAULT CURRENT_DATE,
  receipt_url TEXT,
  is_billable BOOLEAN DEFAULT FALSE,
  is_reimbursable BOOLEAN DEFAULT FALSE,
  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'approved', 'rejected')),
  approved_by UUID REFERENCES public.users(id),
  created_by UUID NOT NULL REFERENCES public.users(id),
  created_at TIMESTAMPTZ DEFAULT NOW()

```

```
);  
CREATE INDEX idx_expenses_org ON public.expenses(organization_id);  
CREATE INDEX idx_expenses_date ON public.expenses(organization_id, expense_date DESC);  
CREATE INDEX idx_expenses_cat ON public.expenses(organization_id, category);
```

7. Multi-Tenant SaaS Architecture

Organization-level isolation, RLS, billing ownership, and RBAC

Tenant Isolation Model

Every record in the database is scoped to an **organization_id**. Row-Level Security (RLS) policies on Supabase enforce that users can only read/write data belonging to organizations they are members of. This provides hard tenant isolation without schema-per-tenant complexity.

Permissions Matrix (RBAC)

Permission	Owner	Admin	Accountant	Manager	Employee
Manage billing & subscription	■	■	■	■	■
Invite / remove members	■	■	■	■	■
Create & send invoices	■	■	■	■	■
Approve expenses	■	■	■	■	■
Submit expenses	■	■	■	■	■
View all reports	■	■	■	■	■
View own expenses only	■	■	■	■	■
Manage chart of accounts	■	■	■	■	■
Delete records	■	■	■	■	■
Access AI CFO (full)	■	■	■	■	■
Export data	■	■	■	■	■
Manage integrations	■	■	■	■	■

8. Supabase Architecture & RLS Policies

Auth, Storage, Edge Functions, Realtime, and Row-Level Security

Supabase Services Map

Service	Usage
Auth	Email/password, Google OAuth, magic links. JWT with org claims injected via custom hook.
PostgreSQL + RLS	All data with org-scoped Row-Level Security. No data leaks between tenants.
Storage	Receipt images, invoice PDFs, avatars. Buckets: receipts/, invoices/, avatars/ with signed URLs.
Edge Functions	Stripe webhooks, AI credit deduction, PDF generation, OCR processing, cron jobs.
Realtime	Live dashboard updates (new expenses, invoice status changes, notifications).
Vault	Encrypted secrets: Stripe keys, Gemma API keys (Google AI Studio), Plaid credentials per org.

Core RLS Policies

```
-- Enable RLS on all tenant tables
ALTER TABLE public.expenses ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.invoices ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.revenues ENABLE ROW LEVEL SECURITY;

-- Helper: check membership
CREATE OR REPLACE FUNCTION public.is_member(org_id UUID)
RETURNS BOOLEAN AS $$
SELECT EXISTS (
  SELECT 1 FROM public.memberships
  WHERE organization_id = org_id
  AND user_id = auth.uid()
);
$$ LANGUAGE SQL SECURITY DEFINER STABLE;

-- Helper: check role
CREATE OR REPLACE FUNCTION public.has_role(org_id UUID, min_role TEXT)
RETURNS BOOLEAN AS $$
SELECT EXISTS (
  SELECT 1 FROM public.memberships
  WHERE organization_id = org_id
  AND user_id = auth.uid()
  AND role = ANY(
    CASE min_role
    WHEN 'owner' THEN ARRAY['owner']
```

```
WHEN 'admin' THEN ARRAY['owner','admin']
WHEN 'accountant' THEN ARRAY['owner','admin','accountant']
WHEN 'manager' THEN ARRAY['owner','admin','accountant','manager']
ELSE ARRAY['owner','admin','accountant','manager','employee']
END
)
);
$$ LANGUAGE SQL SECURITY DEFINER STABLE;

-- Expenses: members can read; employee sees own only
CREATE POLICY "expenses_select" ON public.expenses FOR SELECT USING (
has_role(organization_id, 'manager') OR
(is_member(organization_id) AND created_by = auth.uid())
);
CREATE POLICY "expenses_insert" ON public.expenses FOR INSERT
WITH CHECK (is_member(organization_id));
CREATE POLICY "expenses_update" ON public.expenses FOR UPDATE USING (
has_role(organization_id, 'manager')
);
CREATE POLICY "expenses_delete" ON public.expenses FOR DELETE USING (
has_role(organization_id, 'admin')
);
```

9. Backend Architecture — Node.js / Express

Scalable, layered architecture with separation of concerns

Folder Structure

```
src/  
  ■■■ config/  
  ■ ■■■ supabase.ts # Supabase admin client  
  ■ ■■■ stripe.ts # Stripe SDK init  
  ■ ■■■ env.ts # Zod-validated env vars  
  ■■■ middleware/  
  ■ ■■■ auth.ts # JWT validation + org context  
  ■ ■■■ rbac.ts # Role-based guard factory  
  ■ ■■■ rateLimit.ts # Per-user + per-org rate limits  
  ■ ■■■ validate.ts # Zod request validation  
  ■ ■■■ errorHandler.ts # Centralized error formatter  
  ■■■ routes/  
  ■ ■■■ auth.routes.ts  
  ■ ■■■ organizations.routes.ts  
  ■ ■■■ expenses.routes.ts  
  ■ ■■■ invoices.routes.ts  
  ■ ■■■ revenues.routes.ts  
  ■ ■■■ reports.routes.ts  
  ■ ■■■ billing.routes.ts  
  ■ ■■■ ai.routes.ts  
  ■■■ controllers/ # HTTP layer – thin, delegates to services  
  ■■■ services/ # Business logic  
  ■ ■■■ expense.service.ts  
  ■ ■■■ invoice.service.ts  
  ■ ■■■ report.service.ts  
  ■ ■■■ billing.service.ts  
  ■ ■■■ ai.service.ts  
  ■■■ repositories/ # DB queries (Supabase client calls)  
  ■■■ ai/  
  ■ ■■■ prompts/ # System prompts per feature  
  ■ ■■■ chains/ # LangChain chains  
  ■ ■■■ embeddings.ts # pgvector operations  
  ■ ■■■ categorizer.ts # Expense auto-categorization  
  ■■■ jobs/ # BullMQ background jobs  
  ■ ■■■ invoiceReminder.job.ts
```

```
■ ■■■ cashForecast.job.ts
■ ■■■ reportGeneration.job.ts
■■■ utils/
■■■ pdf.ts # Invoice PDF generation
■■■ logger.ts # Pino structured logging
■■■ currency.ts # FX conversion utilities
```

Architecture Principles

Controller → Service → Repository: HTTP concerns stay in controllers. Business logic in services. All DB access via repositories. Zero DB calls in controllers.

Tenant Context Middleware: Every authenticated request extracts `org_id` and user role into `req.org`. All downstream operations use this context.

Zod Validation: Every request body and query is validated with Zod schemas before hitting the controller.

Background Jobs (BullMQ + Redis): Long-running tasks (PDF generation, AI forecasting, bulk emails) are queued — never block HTTP response.

Structured Logging (Pino): Every request logs: `userId`, `orgId`, `route`, `duration`, `status`. Shipped to Logflare or Datadog.

Error Boundary: A single errorHandler middleware catches all errors, logs them, and returns a consistent JSON error format.

10. REST API Design

Complete endpoint reference with auth, rate limits, and purpose

Method	Route	Purpose	Auth	Rate Limit
POST	/auth/signup	Register new user	Public	10/min
POST	/auth/login	Email/password login	Public	10/min
POST	/auth/logout	Invalidate session	JWT	60/min
POST	/auth/refresh	Refresh JWT token	Refresh token	30/min
GET	/auth/me	Get current user profile	JWT	60/min
POST	/organizations	Create new organization (tenant)	JWT	5/min
GET	/organizations/:id	Get org details	JWT+Member	60/min
PATCH	/organizations/:id	Update org settings	JWT+Admin	20/min
DELETE	/organizations/:id	Delete org (owner only)	JWT+Owner	2/min
POST	/organizations/:id/invite	Invite member by email	JWT+Admin	20/min
GET	/organizations/:id/members	List members and roles	JWT+Member	60/min
GET	/dashboard	KPIs: revenue, expenses, invoices summary	JWT+Member	60/min
GET	/dashboard/cashflow	Monthly cash flow chart data	JWT+Member	60/min
GET	/expenses	List expenses (paginated, filterable)	JWT+Member	120/min
POST	/expenses	Create expense + upload receipt	JWT+Member	60/min
GET	/expenses/:id	Get single expense	JWT+Member	120/min
PATCH	/expenses/:id	Update expense	JWT+Manager	60/min
DELETE	/expenses/:id	Delete expense	JWT+Admin	30/min
POST	/expenses/:id/approve	Approve expense	JWT+Manager	60/min
GET	/invoices	List invoices with filters	JWT+Member	120/min
POST	/invoices	Create invoice	JWT+Accountant	30/min
GET	/invoices/:id	Get invoice detail	JWT+Member	120/min
PATCH	/invoices/:id	Update invoice	JWT+Accountant	30/min
POST	/invoices/:id/send	Send invoice via email	JWT+Accountant	10/min
POST	/invoices/:id/pdf	Generate/download PDF	JWT+Member	20/min
POST	/invoices/:id/mark-paid	Mark invoice as paid	JWT+Accountant	30/min
GET	/reports/pl	Profit & Loss statement	JWT+Accountant	20/min
GET	/reports/cashflow	Cash flow statement	JWT+Accountant	20/min
GET	/reports/balance-sheet	Balance sheet	JWT+Accountant	20/min
GET	/reports/tax-summary	Tax liability summary	JWT+Accountant	10/min

Method	Route	Purpose	Auth	Rate Limit
POST	/reports/export	Export report as PDF/CSV	JWT+Accountant	10/min
POST	/ai/chat	Send message to AI CFO	JWT+Member	20/min
GET	/ai/chat/history	Get conversation history	JWT+Member	30/min
POST	/ai/categorize	Auto-categorize expense	JWT+Member	60/min
GET	/ai/forecast	Get 90-day cash forecast	JWT+Growth+	10/min
GET	/ai/health-score	Get business health score	JWT+Member	5/min
GET	/billing/plans	List available plans	Public	60/min
POST	/billing/subscribe	Create Stripe subscription	JWT+Owner	5/min
POST	/billing/upgrade	Upgrade/downgrade plan	JWT+Owner	5/min
POST	/billing/cancel	Cancel subscription	JWT+Owner	3/min
GET	/billing/invoices	List Stripe billing invoices	JWT+Owner	30/min
POST	/webhooks/stripe	Stripe webhook receiver	Stripe-Sig	Unlimited

11. Frontend Architecture — Next.js

App Router, TypeScript, Tailwind CSS, component-driven design

Page Structure

Page / Route	Description
/ (Landing)	Marketing page with hero, features, pricing CTA, testimonials. SEO optimized.
/pricing	Interactive pricing cards with toggle annual/monthly. Comparison table. Upgrade CTA.
/auth/login	Email + Google OAuth. Clean minimal form. Redirect to onboarding if new user.
/auth/signup	Registration with org creation step. Onboarding wizard (5 steps).
/app/dashboard	KPI cards + sparklines. Recent transactions. AI health score widget. Notifications.
/app/expenses	Data table with filters. Bulk actions. Inline receipt preview. Add expense drawer.
/app/revenue	Income tracking table. Source breakdown chart. YoY comparison.
/app/invoices	Invoice list with status badges. Create invoice flow. PDF preview. Send modal.
/app/reports	P&L, Cash Flow, Balance Sheet tabs. Date range picker. Export button.
/app/ai	Full-page chat interface. AI CFO with financial context. Suggested queries sidebar.
/app/billing	Current plan display. Usage meters. Upgrade/downgrade flow. Invoice history.
/app/settings	Org settings, members, roles, integrations, notifications, security (2FA).
/app/onboarding	Guided 5-step setup: org info → chart of accounts → first invoice → invite → done.

Tech Choices

Concern	Solution
State Management	Zustand for global UI state; React Query (TanStack) for server state + caching
Forms	React Hook Form + Zod resolver — same schemas as backend for consistency
Charts	Recharts (lightweight) for dashboards; Tremor components for KPI cards
Tables	TanStack Table v8 — virtualized, sortable, filterable with server-side pagination
UI Components	shadcn/ui + Radix UI primitives + Tailwind — custom design system on top
PDF Preview	react-pdf for in-browser invoice preview before sending
Notifications	Supabase Realtime subscription → toast notifications via react-hot-toast
File Upload	Supabase Storage direct upload with presigned URLs — progress tracking
AI Chat	Streaming responses via Server-Sent Events; markdown rendering via react-markdown

12. Security & Compliance

Fintech-grade security architecture from day one

Security Controls Matrix

Control	Implementation	Priority
JWT Authentication	Supabase JWTs with 1hr expiry + refresh tokens. Custom claims include org_id and role.	Critical
Row-Level Security	Supabase RLS on every tenant table — enforced at database layer, not just API.	Critical
RBAC Middleware	Express middleware validates role on every protected route before controller executes.	Critical
Rate Limiting	Per-user (IP + JWT) rate limits via Redis. Tiered by plan — paid users get higher limits.	High
Input Validation	Zod schemas on every request body. All user input sanitized before DB insertion.	High
SQL Injection	Supabase client uses parameterized queries exclusively. No raw SQL with user input.	Critical
XSS Prevention	React escapes all output by default. DOMPurify for any HTML rendering. CSP headers.	High
CSRF Protection	SameSite=Strict cookies + CSRF token for sensitive mutations (billing, delete).	High
Secrets Management	All secrets in environment variables. Production secrets in Supabase Vault. Never in code.	Critical
Encryption at Rest	Supabase default AES-256 encryption for all stored data including receipts.	Critical
Encryption in Transit	TLS 1.3 enforced. HSTS headers. All API communication over HTTPS only.	Critical
Audit Logs	Immutable audit_logs table records every write operation with actor, action, timestamp.	High
PII Handling	Email, tax IDs encrypted at rest. Data deletion on org cancel (GDPR compliance).	High
Stripe Security	Webhook signature verification. PCI-DSS delegated to Stripe. No card data stored.	Critical
Dependency Scanning	Snyk + npm audit in CI/CD pipeline. Dependabot auto-PRs for vulnerabilities.	Med

Compliance Considerations

- GDPR (EU): Data portability endpoint, right to deletion, DPA template for B2B customers
- SOC 2 Type I: Leverage Supabase's SOC 2 certification. Implement access controls and audit logs
- PCI-DSS: Fully delegated to Stripe — zero card data touches your servers
- CCPA (California): Privacy policy, opt-out of data sale, data deletion within 45 days
- HIPAA: Not applicable for MVP — flag if healthcare clients emerge in growth phase

13. 4-Week Development Roadmap — Gemma AI Integrated

Strict 30-day execution plan for a 3-person startup team

Week 1: Foundation & Infrastructure

Mon	Project setup: GitHub repo, monorepo structure, ESLint/Prettier, Husky pre-commit
Mon	Supabase project creation: auth config, storage buckets, initial migration
Tue	Database schema: all core tables + RLS policies + helper functions
Tue	Environment setup: Docker Compose (local dev), Railway staging, Vercel preview
Wed	Authentication: Supabase Auth + custom JWT claims hook + user profile sync
Wed	Organization creation flow + membership system + role assignment
Thu	Backend skeleton: Express + TypeScript + Zod + error handler + logger
Thu	Auth middleware + RBAC guards + rate limiter (Redis via Upstash)
Fri	CI/CD: GitHub Actions — lint, test, build, deploy to staging
Fri	Week 1 review: DB migrations stable, auth working end-to-end

Deliverable: Deliverable: Working auth, org creation, DB schema, CI/CD pipeline. Staging environment live.
Team: Full-Stack Engineer: DB + Backend. Designer: Design system setup. PM: User story backlog.

Week 2: Core APIs, Billing & First Gemma Integration

Mon	Expense API: CRUD + receipt upload to Supabase Storage + category validation
Mon	Revenue API: income tracking CRUD + currency support
Tue	Invoice API: create, update, send (Resend email), mark-paid, PDF generation
Tue	Stripe integration: plans sync, subscription creation, webhook handler
Wed	Gemma setup: google-generativeai SDK, env config, rate-limit wrapper, error handler
Wed	Expense categorizer (Gemma): few-shot prompt + org chart-of-accounts context injection
Thu	Dashboard API: aggregate KPIs query (revenue, expenses, outstanding, net)
Thu	Reports API: P&L; and Cash Flow statement generation queries + usage metrics tracking
Fri	Notification system: in-app notifications table + Supabase Realtime push
Fri	Week 2 review: All core CRUD APIs + billing + Gemma categorizer end-to-end tested.

Deliverable: Deliverable: Full REST API + Stripe billing + Gemma expense categorization live on staging.
Team: Engineer: Backend + Gemma integration. Designer: Wireframes for all pages. PM: Stripe test scenarios.

Week 3: Frontend & Dashboard

Mon	Next.js setup: App Router, auth middleware, org context provider, route protection
Mon	Landing page + pricing page (static, SEO optimized)
Tue	Auth pages: login, signup, email verify. Supabase client integration.
Tue	Onboarding wizard: 5-step org setup flow with progress indicator
Wed	Dashboard: KPI cards, cash flow chart (Recharts), recent transactions, health score
Wed	Expenses page: TanStack Table, filters, add drawer, receipt upload, approve flow
Thu	Invoices page: list, create flow (multi-step), PDF preview, send modal
Thu	Reports page: P&L; table, cash flow chart, date range picker, CSV/PDF export
Fri	Billing page: plan cards, usage meters, upgrade modal, Stripe Customer Portal
Fri	Settings: org settings, member management, notification preferences

Deliverable: Deliverable: Fully functional frontend for all MVP pages. Mobile-responsive. Connected to API.
Team: Engineer: Frontend integration. Designer: Pixel-perfect implementation review. PM: UAT testing.

Week 4: Gemma AI Features, Testing & Launch

Mon	Gemma NL search: prompt engineering for query→JSON filter, API endpoint, search UI bar
Mon	Business health score: DB scoring cron + Gemma narrative endpoint + dashboard widget
Tue	Onboarding wizard AI: Gemma recommends chart-of-accounts based on 5 user answers
Tue	Vendor name normalization: Gemma batch job cleans imported transaction descriptions
Wed	End-to-end testing: signup → categorize expense → send invoice → NL search → report
Wed	Performance: query optimization, DB indexes review, Gemma response caching (Redis)
Thu	Security audit: RLS policy testing, JWT edge cases, Stripe webhook + Gemma input validation
Thu	Bug bash: fix all P0/P1 issues found in testing. Load test with k6.
Fri	Production deployment: Vercel + Railway. DNS setup. Monitoring (Sentry + Logflare).
Fri	LAUNCH: Activate waitlist users. Send launch emails. Monitor dashboards + Gemma costs.

Deliverable: Deliverable: Production app live with Gemma AI categorization, NL search, health score, and onboarding AI.
Team: Engineer: Gemma prompts + deployment. Designer: Final polish + launch assets. PM: Launch coordination.

14. DevOps & Deployment

CI/CD, Docker, monitoring, logging, alerting, and disaster recovery

Infrastructure Stack

Component	Tool	Notes
Frontend Hosting	Vercel	Global CDN, preview deployments per PR, Edge Functions
Backend Hosting	Railway	Dockerized Node.js, auto-scaling, private networking
Database	Supabase (managed Postgres)	Daily backups, PITR, connection pooling (PgBouncer)
Cache / Queues	Upstash Redis	Rate limiting, BullMQ job queue, session cache
File Storage	Supabase Storage (S3-compatible)	Receipts, PDFs, avatars with CDN delivery
Email	Resend	Invoice delivery, notifications, transactional emails
Monitoring	Sentry	Error tracking, performance monitoring, release tracking
Logging	Logflare (Supabase native)	Structured logs, log search, retention 30 days
Alerting	PagerDuty + Slack webhooks	On-call alerts for 5xx spikes, DB connections, latency
CI/CD	GitHub Actions	Lint → Test → Build → Deploy (staging then prod)

CI/CD Pipeline

```
# .github/workflows/deploy.yml (simplified)
name: CI/CD Pipeline
on:
  push:
    branches: [main, staging]
  pull_request:
    branches: [main]
jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm run lint
      - run: npm run type-check
      - run: npm run test -- --coverage
```

```
deploy-staging:
needs: quality
if: github.ref == 'refs/heads/staging'
steps:
- run: railway up --environment staging

deploy-production:
needs: quality
if: github.ref == 'refs/heads/main'
environment: production # requires manual approval
steps:
- run: supabase db push # run migrations
- run: railway up --environment production
- run: vercel --prod
```

15. Scaling Strategy

From 100 users to 1M users — incremental architecture evolution

Scale	Architecture Changes	Infra Cost	Key Action
100 Users	Current setup sufficient. Supabase free/pro tier. Single Railway instance. No optimization needed.	\$0-50/mo	Monitor, don't optimize
1,000 Users	Add Upstash Redis caching for dashboard queries. Optimize slow DB queries. Add Supabase read replica.	\$150-300/mo	Cache hot queries
10,000 Users	Horizontal scaling on Railway (2-3 instances). BullMQ for all async work. CDN for static assets. DB connection pooling tuned.	\$800-1,500/mo	Queue everything async
100,000 Users	Dedicated Postgres (AWS RDS or Supabase Pro). Separate AI service. Redis Cluster. Background workers separate fleet.	\$5,000-15,000/mo	Split services
1,000,000 Users	Multi-region deployment. Read replicas per region. Kafka for event streaming. Microservices for billing, AI, notifications.	\$50,000+/mo	True distributed system

Caching Strategy

- Dashboard KPIs: Cache per org for 5 minutes — most users don't need real-time totals
- Report generation: Cache P&L/Cash Flow for 1 hour — trigger invalidation on new transaction
- AI responses: Cache identical queries per org for 24 hours — 70% of queries are repeated
- Plan/feature flags: Cache in Redis with 1-hour TTL — checked on every API request
- Exchange rates: Cache FX rates for 1 hour from external API

16. Launch Strategy

Beta, waitlist, onboarding, activation, retention, and referral

Pre-Launch (Week -2 to 0)

- Build waitlist landing page with Webflow or Framer — collect emails with 'Join Beta' CTA
- Post to Twitter/X: Build in public thread — show dashboard screenshots weekly during development
- Submit to Product Hunt upcoming page — build upvote momentum before launch day
- LinkedIn outreach to 50 target accounts (freelancers, startup founders, accountants)
- Create content: '5 ways AI will replace your accountant' — rank for accounting SaaS keywords
- Set up Intercom or Crisp for in-app support chat — critical for onboarding first users
- Partner with 3-5 accounting influencers on YouTube/LinkedIn for sponsored reviews at launch

Launch Day (Day 1)

- Post on Product Hunt at 12:01 AM PST — all team members and friends upvote and comment
- Send launch email to full waitlist — 'You're in! FinanceAI is live' with 3-month 50% discount code
- Tweet storm: features thread, demo GIF, pricing, link
- Post in 10 Slack communities: Indie Hackers, Founder's Network, SaaS Alliance, etc.
- Run \$500 LinkedIn ads targeting 'small business owner' + 'freelancer' + 'startup founder'
- Monitor Sentry, Railway logs, and Supabase dashboard every 30 minutes — fix issues fast

Activation Metrics (North Star)

Metric	Target	Action if Missed
Sign-up to first expense logged	< 5 minutes	If > 10 min, fix onboarding UX
Sign-up to first invoice sent	< 24 hours	Track in Mixpanel, add in-app nudges
Day 7 retention rate	> 40%	Email reminders if user goes cold after day 3
Free-to-paid conversion	> 8%	Trigger upgrade prompt when user hits 80% of quota
Monthly churn rate	< 3%	NPS survey at day 14, proactive outreach to at-risk accounts
NPS Score	> 50	Run survey at day 30 for all active users

Referral System

Every user gets a unique referral link. When a referred user upgrades to a paid plan, the referrer earns **1 free month** added to their subscription. Display referral stats on the billing page. Top referrers (5+) get permanent 30% discount — turns power users into unpaid sales reps. Send monthly 'You earned a free month!' emails to keep the loop viral.

Final Word: The businesses that win in SaaS don't build the most features — they build the fastest path to value. Ship the core loop in 30 days: signup → log expense → send invoice → see P&L.; Then let the AI surprise users. Every hour not shipping is an hour a competitor is gaining. Build fast. Measure obsessively. Iterate relentlessly.

Ready to Build?

This blueprint gives you everything needed to launch in 30 days.

Start with Week 1, Day 1: `supabase init`

FinanceAI · MVP Blueprint · v2.0 · Gemma AI Edition · 2026